

Automata and Computability

A Complete Guide

From Finite Automata to the Limits of Computation

Based on Sipser's *Introduction to the Theory of Computation*

Contents

1	Why This Topic Exists	2
2	Mathematical Foundations	2
2.1	Alphabets, Strings, and Languages	2
3	Finite Automata — The Simplest Machines	2
3.1	Deterministic Finite Automata (DFA)	2
3.2	Nondeterministic Finite Automata (NFA)	3
3.3	Regular Languages	3
3.4	Regular Expressions	3
4	The Pumping Lemma — Proving Non-Regularity	3
5	Context-Free Grammars and Pushdown Automata	4
5.1	Context-Free Grammars (CFG)	4
5.2	Parse Trees and Ambiguity	4
5.3	Pushdown Automata (PDA)	4
5.4	Pumping Lemma for CFLs	4
6	Turing Machines — The Universal Model	5
6.1	Definition	5
6.2	Church–Turing Thesis	5
6.3	Recognizers vs. Deciders	5
6.4	The Universal Turing Machine	5
7	Decidability and Undecidability	5
7.1	The Halting Problem	6
7.2	Rice's Theorem	6
7.3	Other Undecidable Problems	6
8	Reducibility	6
9	The Chomsky Hierarchy	7
10	Complexity Theory	7
10.1	Time Complexity	7
10.2	The P vs. NP Question	7
10.3	NP-Completeness	7
10.4	Space Complexity	8
11	Key Theorems Summary	8
12	Further Reading	8

1 Why This Topic Exists

Before building computers, mathematicians asked: **what does it mean to “compute” something?** Alan Turing, Alonzo Church, and others in the 1930s formalized this. Their work led to the modern computer — but also proved that some problems are *forever unsolvable* by any algorithm. Automata theory gives us the mathematical machinery to reason about this rigorously.

2 Mathematical Foundations

2.1 Alphabets, Strings, and Languages

Definition 2.1: Alphabet, String, Language

- An **alphabet** Σ is a finite, non-empty set of symbols, e.g. $\Sigma = \{0, 1\}$.
- A **string** is a finite sequence of symbols from Σ , e.g. 0110.
- The **empty string** ε is a string of length zero.
- Σ^* denotes the set of all possible strings over Σ (including ε).
- A **language** L is any subset of Σ^* .

Languages are the central objects of study. A “problem” is often phrased as: *does a given string belong to some language?*

3 Finite Automata — The Simplest Machines

3.1 Deterministic Finite Automata (DFA)

A DFA is the simplest computational model — a machine with a **finite amount of memory** (its states).

Definition 3.1: Deterministic Finite Automaton

A DFA is a 5-tuple $M = (Q, \Sigma, \delta, q_0, F)$ where:

- Q — finite set of **states**
- Σ — input **alphabet**
- $\delta : Q \times \Sigma \rightarrow Q$ — **transition function**
- $q_0 \in Q$ — **start state**
- $F \subseteq Q$ — set of **accept states**

The machine reads input left to right, one symbol at a time. After reading all input: if in an accept state $\Rightarrow$ **accept**; otherwise $\Rightarrow$ **reject**.

Example 3.1: DFA for binary strings ending in 1

States: $\{q_0, q_1\}$; Start: q_0 ; Accept: $\{q_1\}$

$$\delta(q_0, 0) = q_0, \quad \delta(q_0, 1) = q_1, \quad \delta(q_1, 0) = q_0, \quad \delta(q_1, 1) = q_1$$

3.2 Nondeterministic Finite Automata (NFA)

An NFA relaxes the DFA: at any point it can *branch* into multiple states simultaneously, and can take ε -transitions (move without reading input).

Definition 3.2: Nondeterministic Finite Automaton

Like a DFA, but:

$$\delta : Q \times (\Sigma \cup \{\varepsilon\}) \longrightarrow \mathcal{P}(Q)$$

The transition function returns a *set* of states. An NFA accepts if **any** branch of computation reaches an accept state.

Theorem 3.1: DFA–NFA Equivalence

Every NFA can be converted to an equivalent DFA via the *subset construction*. Therefore, NFAs and DFAs recognize exactly the same class of languages — though the DFA may have exponentially more states.

3.3 Regular Languages

Definition 3.3: Regular Language

A language is **regular** if it is recognized by some DFA (equivalently, some NFA, or some regular expression).

Regular languages are closed under union, concatenation, Kleene star, complement, and intersection.

3.4 Regular Expressions

Regular expressions are an algebraic way to *describe* regular languages using union ($\cup$), concatenation ($\circ$), and Kleene star ($*$). By Kleene's theorem, every regular expression describes a regular language and vice versa.

4 The Pumping Lemma — Proving Non-Regularity

Theorem 4.1: Pumping Lemma for Regular Languages

If L is regular, then there exists a pumping length $p \geq 1$ such that any string $s \in L$ with $|s| \geq p$ can be split as $s = xyz$ satisfying:

1. $|y| > 0$
2. $|xy| \leq p$
3. For all $i \geq 0$, $xy^iz \in L$

Example 4.1: $L = \{0^n 1^n \mid n \geq 0\}$ is not regular

Assume L is regular with pumping length p . Take $s = 0^p 1^p$. Any valid split xyz with $|xy| \leq p$ forces y to consist only of 0s. Pumping y (i.e. taking $i \neq 1$) produces more

0s than 1s, a contradiction.

5 Context-Free Grammars and Pushdown Automata

5.1 Context-Free Grammars (CFG)

Definition 5.1: Context-Free Grammar

A CFG is a 4-tuple $G = (V, \Sigma, R, S)$ where:

- V — finite set of **variables** (non-terminals)
- Σ — **terminals**
- R — finite set of **production rules** $A \rightarrow w$ with $A \in V$, $w \in (V \cup \Sigma)^*$
- $S \in V$ — **start variable**

A string is in the language if it can be derived from S by repeatedly applying rules.

Example 5.1: Grammar for $\{0^n 1^n \mid n \geq 0\}$

$S \rightarrow 0S1 \mid \varepsilon$

This generates: ε , 01, 0011, 000111, ...

5.2 Parse Trees and Ambiguity

A **parse tree** visualizes how a grammar derives a string. A grammar is **ambiguous** if some string has more than one parse tree — this causes problems in programming language design.

5.3 Pushdown Automata (PDA)

A PDA is an NFA augmented with a **stack**. This extra power is precisely what is needed to recognize context-free languages.

Theorem 5.1: CFG–PDA Equivalence

A language is context-free if and only if some PDA recognizes it.

Context-free languages are closed under union, concatenation, and Kleene star, but **not** under intersection or complement.

5.4 Pumping Lemma for CFLs

Theorem 5.2: Pumping Lemma for Context-Free Languages

If L is context-free, there exists a pumping length p such that any $s \in L$ with $|s| \geq p$ can be split as $s = uvxyz$ with:

1. $|vy| > 0$
2. $|vxy| \leq p$
3. For all $i \geq 0$, $uv^i xy^i z \in L$

Example 5.2: $L = \{a^n b^n c^n \mid n \geq 0\}$ is not context-free

A CFL can balance two dependent counters but not three simultaneously. Applying the pumping lemma to $a^p b^p c^p$ leads to a contradiction regardless of how vy is chosen.

6 Turing Machines — The Universal Model

6.1 Definition

Definition 6.1: Turing Machine

A TM is a 7-tuple $(Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$ where:

- Q — finite set of states
- Σ — input alphabet ($\sqcup \notin \Sigma$)
- Γ — tape alphabet ($\sqcup \in \Gamma, \Sigma \subseteq \Gamma$)
- $\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$ — transition function
- q_0 — start state; $q_{\text{accept}}, q_{\text{reject}}$ — halting states

The tape is infinite. The head reads, writes, and moves left or right on each step.

6.2 Church–Turing Thesis

Key Result 6.1: Church–Turing Thesis

Any function that is intuitively computable can be computed by a Turing Machine. This thesis is not formally provable but is universally accepted, and defines what “computable” means.

6.3 Recognizers vs. Deciders

Definition 6.2: Recognizer and Decider

- A TM **recognizes** L if it accepts all strings in L (but may loop on strings not in L).
- A TM **decides** L if it always halts — accepts strings in L , rejects strings outside L .
- A language is **Turing-recognizable** (r.e.) if some TM recognizes it.
- A language is **decidable** (recursive) if some TM decides it.

6.4 The Universal Turing Machine

There exists a TM $\mathcal{U}$ that takes as input a description $\langle M \rangle$ of another TM and an input w , and simulates M on w . This is the theoretical foundation of the modern stored-program computer.

7 Decidability and Undecidability

7.1 The Halting Problem

$$\text{HALT} = \{\langle M, w \rangle \mid \text{TM } M \text{ halts on input } w\}$$

Theorem 7.1: Undecidability of the Halting Problem (Turing, 1936)

HALT is **undecidable**.

Proof sketch (diagonalization): Suppose a decider H for HALT exists. Construct machine D :

- D takes input $\langle M \rangle$ and runs H on $\langle M, \langle M \rangle \rangle$.
- If H says “halts” $\Rightarrow D$ loops forever.
- If H says “loops” $\Rightarrow D$ halts and accepts.

Ask: what does D do on input $\langle D \rangle$?

- D halts on $\langle D \rangle \Rightarrow H$ says “halts” $\Rightarrow D$ loops. **Contradiction.**
- D loops on $\langle D \rangle \Rightarrow H$ says “loops” $\Rightarrow D$ halts. **Contradiction.**

Hence H cannot exist. $\square$

7.2 Rice's Theorem

Theorem 7.2: Rice's Theorem

Any non-trivial semantic property of Turing machines is undecidable. In other words, no algorithm can determine whether an arbitrary program satisfies any meaningful behavioral property.

7.3 Other Undecidable Problems

- Does TM M accept string w ? — Undecidable
- Does TM M accept any string at all? — Undecidable
- Post Correspondence Problem — Undecidable
- Hilbert's 10th Problem (integer solutions to Diophantine equations) — Undecidable (Matiyasevich, 1970)

8 Reducibility

Definition 8.1: Mapping Reduction

A is **mapping-reducible** to B (written $A \leq_m B$) if there exists a computable function f such that:

$$w \in A \iff f(w) \in B$$

Theorem 8.1: Reduction and Decidability

1. If $A \leq_m B$ and B is decidable, then A is decidable.
 2. Contrapositive: if A is undecidable and $A \leq_m B$, then B is undecidable.
- This is the primary tool for proving undecidability of new problems.

9 The Chomsky Hierarchy

All formal languages fit into a nested hierarchy:

Class	Machine	Grammar	Example
Regular	DFA / NFA	Regular grammar	$\{0^n \mid n \geq 0\}$
Context-Free	PDA	CFG	$\{0^n 1^n\}$
Context-Sensitive	Linear-bounded TM	CSG	$\{0^n 1^n 2^n\}$
Recursively Enumerable	Turing Machine	Unrestricted	HALT

Each class is *strictly* contained in the next.

10 Complexity Theory

10.1 Time Complexity

Definition 10.1: Complexity Classes P and NP

- $P = \bigcup_k \text{TIME}(n^k)$ — languages decidable in **polynomial time**. These are considered “tractable.”
- **NP** — languages where a given solution can be **verified** in polynomial time (equivalently, solvable by a nondeterministic TM in polynomial time).

10.2 The P vs. NP Question

Key Result 10.1: The Greatest Open Problem

Is $P = NP$?

If yes, every problem whose solution is easy to *check* is also easy to *find*. Most researchers believe $P \neq NP$, but no proof exists. This is the central open question of computer science.

10.3 NP-Completeness

Definition 10.2: NP-Complete

A problem B is **NP-complete** if:

1. $B \in NP$, and
2. Every problem in NP is polynomial-time reducible to B (B is **NP-hard**).

Theorem 10.1: Cook–Levin Theorem (1971)

SAT (Boolean satisfiability) is NP-complete — the first problem proved to be so. If any NP-complete problem is in P, then $P = NP$.

Well-known NP-complete problems include: 3-SAT, Clique, Vertex Cover, Hamiltonian Path, Traveling Salesman (decision), and Graph 3-Coloring.

10.4 Space Complexity

PSPACE is the class of problems solvable with polynomial *space*. We know:

$$P \subseteq NP \subseteq PSPACE$$

but whether any of these containments are strict equalities remains open.

11 Key Theorems Summary

Theorem	Statement
Kleene's Theorem	Regular languages = DFA languages = NFA languages = Regex languages.
Myhill–Nerode	A language is regular iff it has finitely many equivalence classes under indistinguishability.
Church–Turing Thesis	Turing machines capture all effective computation.
Halting Problem	No TM can decide whether an arbitrary TM halts on a given input.
Rice's Theorem	All non-trivial semantic properties of TMs are undecidable.
Cook–Levin	SAT is NP-complete.

12 Further Reading

- **Sipser, M.** *Introduction to the Theory of Computation*, 3rd ed. — the gold standard textbook; beautifully written and self-contained.
- **Arora, S. & Barak, B.** *Computational Complexity: A Modern Approach* — advanced treatment of complexity theory.
- **MIT OpenCourseWare 18.404J** — Theory of Computation (freely available online, taught by Sipser himself).